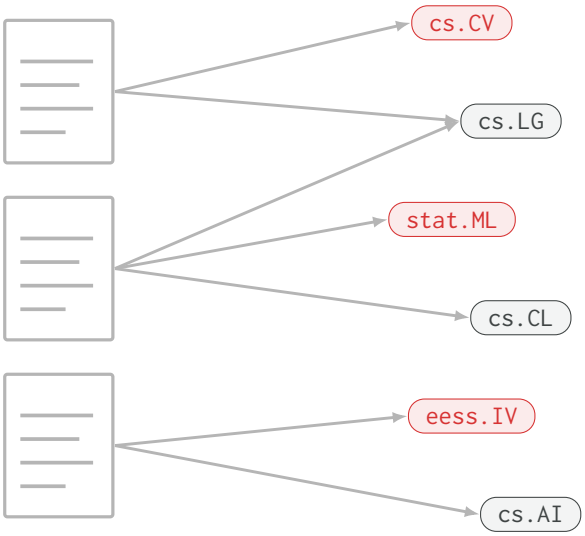
Lingüística Computacional

Máster Universitario en Inteligencia Artificial,
Reconocimiento de Formas e Imagen Digital

Clasificación multietiqueta de artículos científicos

Modelos clásicos y Transformers sobre resúmenes de arXiv

Memoria del trabajo final



AUTORES
Carlos Broch Cubedo
Ferran Artero Merino

EQUIPO EN CODABENCH
LC02

ENTREGA
27 de octubre de 2026

Índice

1 Introducción **1**
- 1.1 Descripción de la tarea . . . 1
- 1.2 Métricas de evaluación . . . 1

2 Datos y preprocesado **3**
- 2.1 Análisis exploratorio . . . 3
- 2.2 Preprocesado y protocolo de validación . . . 4

3 Modelos clásicos **7**
- 3.1 Planteamiento común . . . 7
- 3.2 Búsqueda de hiperparámetros . . . 8
- 3.3 Técnicas para las clases minoritarias . . . 8
- 3.4 Representaciones por debajo de la palabra . . . 9
- 3.5 Modelo final . . . 10

4 Modelos basados en Transformers **12**
- 4.1 Configuración común . . . 12
- 4.2 Representaciones congeladas . . . 12
- 4.3 Ajuste fino de modelos preentrenados . . . 13
- 4.4 Ablaciones . . . 14
- 4.5 Modelos finales . . . 14

5 Combinación de modelos **17**
- 5.1 Protocolo . . . 17
- 5.2 Métodos y grupos de modelos . . . 17
- 5.3 Resultados . . . 18
- 5.4 Resultado en *Codabench* . . . 19

6 Clasificación *few-shot* con un LLM **21**
- 6.1 Configuración . . . 21
- 6.2 Resultados . . . 21

7 Resultados y conclusiones **23**
- 7.1 Resumen de resultados . . . 23
- 7.2 Conclusiones . . . 24

01

Introducción

1.1 Descripción de la tarea

El objetivo es asignar automáticamente categorías temáticas de *arXiv* a artículos científicos a partir de su título y su resumen. Un artículo puede pertenecer a varias áreas a la vez (p. ej., un trabajo de segmentación de imágenes médicas puede etiquetarse como `cs.CV`, `cs.AI` y `cs.LG`), por lo que se trata de un problema de clasificación multietiqueta. Para cada documento se predice un subconjunto, de tamaño variable, de $L = 28$ categorías.

El corpus procede de una colección pública de *arXiv*[1] filtrada a sus categorías más frecuentes. El conjunto de entrenamiento contiene 43.555 artículos descritos por su título, su resumen y su lista de categorías. El de test lo forman 4.885 artículos más recientes, sin etiquetas, con una columna adicional de *keywords* que no existe en entrenamiento.

La tarea se plantea como una competición en *Codabench* con dos sistemas de referencia que hay que superar: un *Naive Bayes* multinomial y un `roberta-base` ajustado para clasificación multietiqueta.

1.2 Métricas de evaluación

Sean N documentos con vectores de referencia $\mathbf{y}_i \in \{0,1\}^L$ y predichos $\hat{\mathbf{y}}_i \in \{0,1\}^L$. La competición usa la exactitud de subconjunto y la *Hamming loss*,

$$\text{Acc} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[\hat{\mathbf{y}}_i = \mathbf{y}_i], \qquad \text{HL} = \frac{1}{NL}\sum_{i=1}^{N}\sum_{j=1}^{L} \mathbf{1}[\hat{y}_{ij} \neq y_{ij}], \tag{1.1}$$

junto con la precisión, la cobertura y la F1 promediadas de forma *macro* sobre las categorías:

$$F1_{\text{macro}} = \frac{1}{L}\sum_{j=1}^{L} \frac{2\,P_j R_j}{P_j + R_j}, \qquad P_j = \frac{TP_j}{TP_j + FP_j}, \quad R_j = \frac{TP_j}{TP_j + FN_j}, \tag{1.2}$$

con `zero_division=0`, de modo que una categoría que nunca se predice cuenta como cero.

Estas métricas no tiran en la misma dirección. La exactitud no da crédito a los aciertos parciales, y la HL es engañosamente baja cuando cada documento tiene pocas etiquetas de las 28 posibles, porque la mayoría de decisiones binarias son negativos triviales.

[1] https://www.kaggle.com/datasets/spsayakpaul/arxiv-paper-abstracts/data

Ambas favorecen predecir pocas etiquetas. La F1 macro, en cambio, pondera por igual todas las categorías con independencia de su frecuencia, y premia así a los sistemas que también aciertan en las clases minoritarias. Por este motivo es el criterio principal para seleccionar modelos, umbrales e hiperparámetros.

02

Datos y preprocesado

2.1 Análisis exploratorio

Antes de entrenar ningún modelo se analizaron el conjunto de entrenamiento y su relación con el de test. Esta sección recoge los hallazgos que condicionan el diseño de los sistemas y la decisión que se deriva de cada uno.

Duplicados. Las 43.555 filas del entrenamiento corresponden a 32.577 artículos únicos. Un total de 8.900 artículos aparecen entre dos y siete veces, de modo que el 25 % de las filas son copias. Solo 6 de estos artículos tienen etiquetas distintas entre copias, y siempre difieren en una única categoría, por lo que el ruido de etiquetado es despreciable. Si no se agrupan las copias, un mismo artículo puede quedar a la vez en entrenamiento y en validación, y las métricas locales saldrían infladas. Por ello, toda partición se hace sobre artículos únicos.

Desequilibrio extremo. La distribución de categorías tiene una cola muy larga (Figura 2.1). La etiqueta `cs.CV` aparece en 19.787 artículos y `physics.chem-ph` en 20, una diferencia de casi mil veces. Solo 6 categorías superan los 1.000 artículos, 12 de las 28 no llegan a 100 y las 5 más frecuentes acumulan el 90 % de las asignaciones de etiqueta. Como la F1 macro pondera por igual todas las categorías, el resultado depende sobre todo de las clases raras. Esto motiva el uso de pesos por clase (`class_weight="balanced"`) y de un umbral de decisión ajustado por clase en lugar del umbral fijo de 0’5. Además, las clases más raras solo tienen entre 4 y 10 positivos en cada partición de validación, por lo que su F1 es muy ruidoso y los resultados se reportan con su desviación entre particiones.

Estructura de las etiquetas. Cada artículo tiene entre 1 y 4 etiquetas (1’75 de media; el 46 % tiene solo una), y en todo el corpus solo aparecen 71 combinaciones distintas. Las etiquetas siguen además una jerarquía muy marcada: el 100 % de los artículos lleva al menos una de tres categorías raíz (`cs.CV`, `cs.LG` o `stat.ML`), que son las únicas que aparecen solas, y el 70’7 % son combinaciones entre estas. Las 25 categorías restantes son secundarias, y la matriz de probabilidades condicionadas (Figura 2.2) muestra que muchas determinan su raíz: `eess.IV`, `cs.GR`, `cs.MM` y `cs.HC` aparecen siempre con `cs.CV`, y `cs.CR`, `cs.SI` o `eess.SP`, siempre con `cs.LG`. La misma figura revela tres parejas que son alias en la taxonomía de *arXiv* y aparecen siempre juntas: `cs.IT`/`math.IT`, `cs.SY`/`eess.SY` y

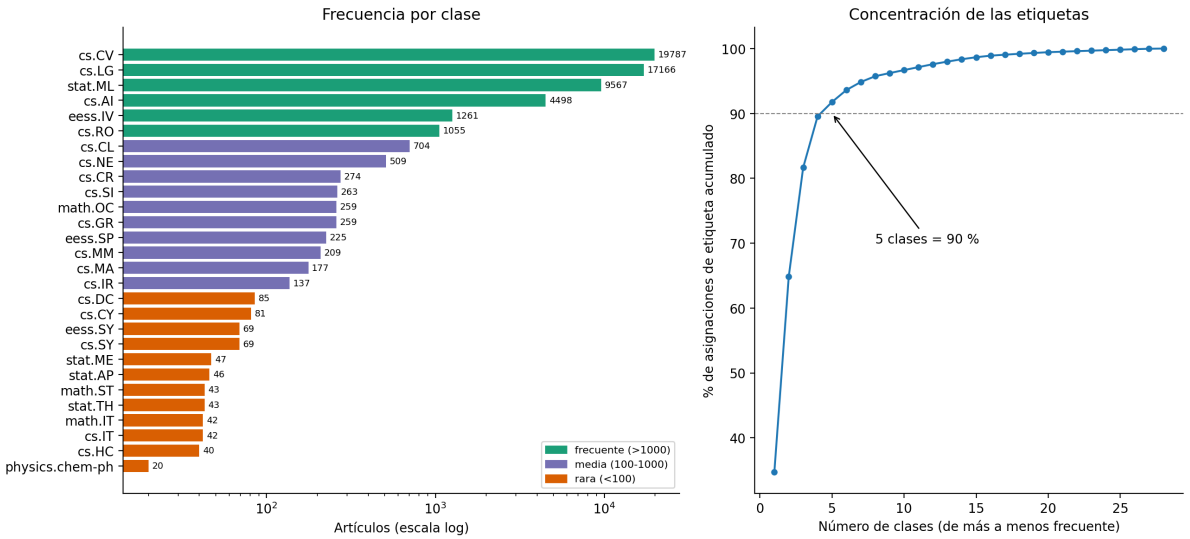


Figura 2.1. Izquierda: artículos únicos por categoría (escala logarítmica). Derecha: porcentaje acumulado de asignaciones de etiqueta al añadir categorías de más a menos frecuente.

`math.ST`/`stat.TH`.

Texto. Los títulos tienen unas 9 palabras y los resúmenes unas 170. El texto apenas tiene ruido, solo saltos de línea y, en un 6–7 % de los resúmenes, fórmulas en LaTeX. Estas se concentran en las clases matemáticas. Concatenar título y resumen da mejor resultado que usar cualquiera de los dos por separado.

Desplazamiento entre entrenamiento y test. El test está formado por artículos más recientes, y la diferencia es grande. Para medirlo se entrena un clasificador (regresión logística sobre TF-IDF) para predecir si un texto procede de entrenamiento o de test, y se evalúa con el área bajo la curva ROC (AUC). El resultado es un AUC de 0'98 (Figura 2.3), es decir, que casi todos los textos de test se reconocen como tales. El 48 % de los tipos de palabra del test no aparece nunca en entrenamiento: términos como *LLM*, *LoRA*, *in-context* o *agentic*. Los resúmenes del test son además un 10 % más largos. Este desplazamiento tiene dos consecuencias. La primera es que las métricas de validación local serán optimistas y sirven para comparar modelos, no para predecir el resultado en *Codabench*. La segunda es que favorece representaciones robustas al vocabulario nuevo, como los n-gramas de caracteres o los modelos preentrenados recientes. La columna *keywords* del test resulta ser un vocabulario cerrado de 29 términos que no existe en entrenamiento, por lo que no se utiliza.

2.2 Preprocesado y protocolo de validación

Todos los sistemas comparten el mismo preprocesado y la misma partición, de modo que sus resultados son directamente comparables.

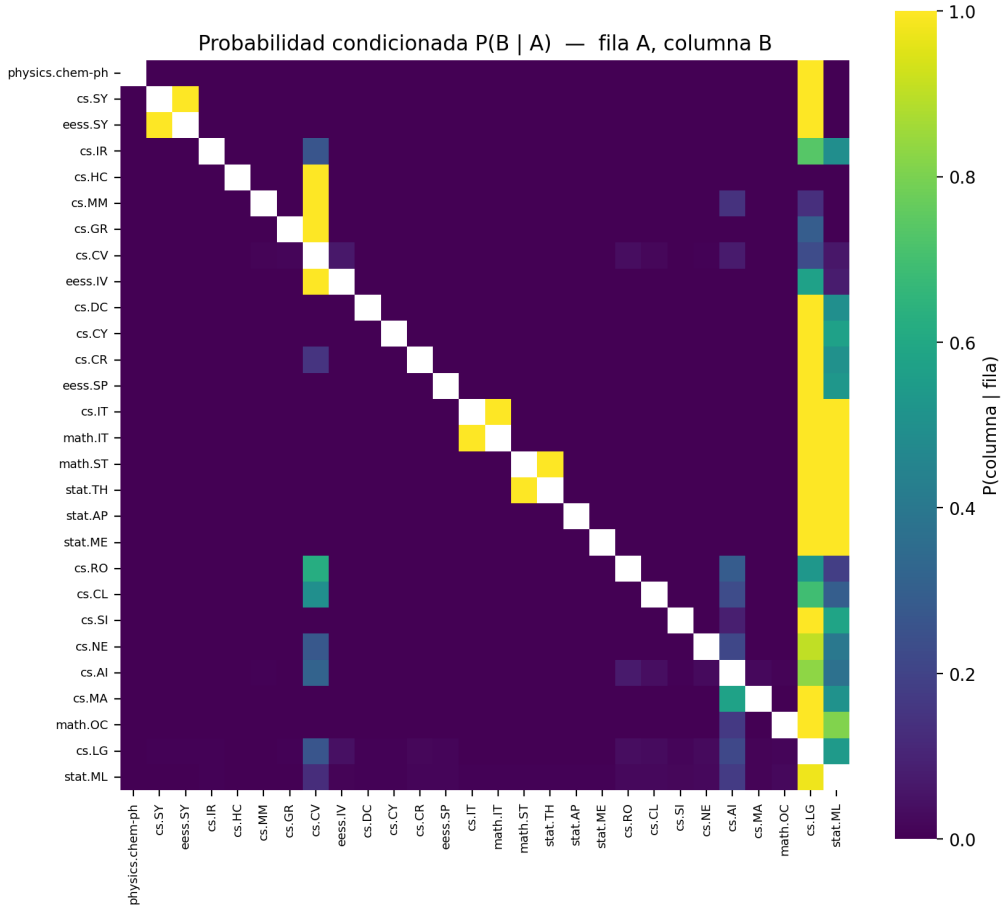


Figura 2.2. Probabilidad de que un artículo de la categoría de la fila lleve también la de la columna, $P(B \mid A)$. Las columnas `cs.LG`, `stat.ML` y `cs.CV` concentran las inclusiones (celdas amarillas), y los pares simétricos junto a la diagonal son los tres alias.

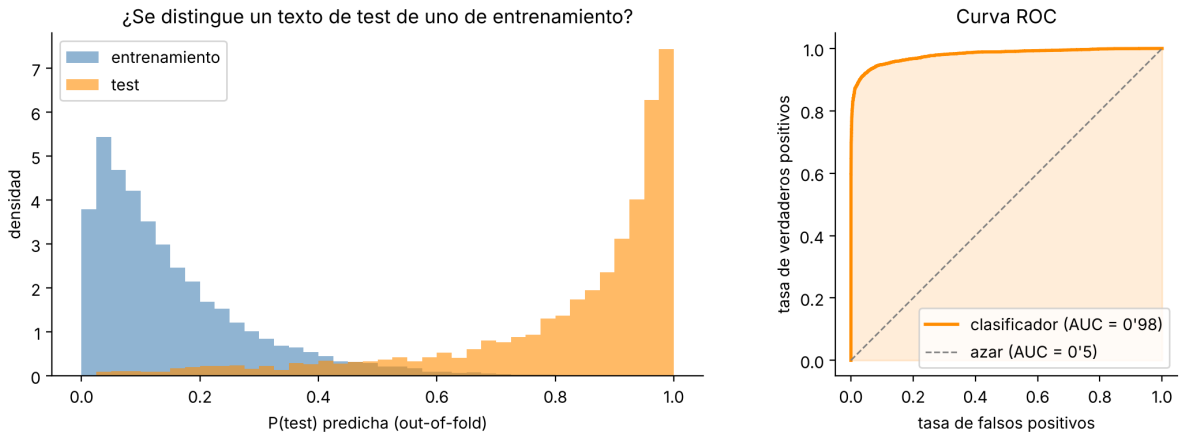


Figura 2.3. Validación adversarial. Izquierda: probabilidad de pertenecer al test que asigna el clasificador (predicciones *out-of-fold*) a textos de entrenamiento y de test. Derecha: curva ROC; la diagonal corresponde a un clasificador al azar.

1. **Agrupación de duplicados.** Las filas se agrupan por título y resumen normalizados (minúsculas y sin puntuación ni LaTeX). Cada artículo único recibe la unión de las etiquetas de sus copias.
2. **Texto de entrada.** `título. resumen`, normalizando solo los espacios. Cualquier otra transformación depende del modelo.
3. **Binarización.** Las etiquetas se codifican como vectores de 28 dimensiones con `MultiLabelBinarizer`.
4. **Partición.** Se reserva un 15 % de los artículos únicos como *holdout*, que no se usa para tomar ninguna decisión. El 85 % restante se divide en 3 particiones (`MultilabelStratifiedKFold`) para la selección de modelos, hiperparámetros y umbrales.

03

Modelos clásicos

La experimentación con modelos clásicos se hizo en tres pasos. Primero se buscaron los mejores hiperparámetros de tres clasificadores lineales sobre TF-IDF. Como el resultado seguía limitado por las clases minoritarias, después se probaron dos técnicas propias para reforzarlas: el deduplicado inteligente y un enfoque jerárquico. Por último, se cambió la representación del texto añadiendo unidades más pequeñas que la palabra. Todos los experimentos comparten el preprocesado y la partición de la Sección 2.2, y la semilla se fija.

3.1 Planteamiento común

Representación. Cada artículo se representa con TF-IDF sobre n-gramas de palabras con normalización L_2, filtrado de frecuencia `min_df=2` y `max_df=0.9` con un máximo de 300.000 características y sin eliminar *stop words*. El vectorizador se ajusta solo con los textos de entrenamiento de cada partición.

Clasificadores. Con `OneVsRestClassifier`, el problema se descompone en 28 problemas binarios independientes (*binary relevance*). Se comparan tres clasificadores:

- ***Naive Bayes*** multinomial, con $\alpha \in \{0.01, 0.03, 0.1, 0.3, 1\}$.
- **Regresión logística** (`liblinear`) con $C \in \{2, 8, 32\}$.
- **SVM lineal** (`LinearSVC`) con $C \in \{0.05, 0.1, 0.3, 1\}$.

La regresión logística y la SVM usan `class_weight="balanced"`, que pondera cada ejemplo de forma inversamente proporcional a la frecuencia de su clase. Cada clasificador produce una puntuación por categoría: la distancia al hiperplano en la SVM y el logaritmo del cociente de probabilidades en los otros dos, cuyo umbral natural es 0 en los tres casos. En *Naive Bayes* no se usa `predict_proba`, porque con textos largos se satura en 0 o 1 y no permite ajustar umbrales.

Umbral por clase. Con el umbral por defecto, los clasificadores de las clases raras casi nunca predicen la clase positiva. Por eso cada categoría j recibe su propio umbral θ_j, el que maximiza su F1 entre 200 cuantiles de sus puntuaciones. Para no evaluar sobre los mismos datos con los que se ajusta, los umbrales se calculan por *cross-fitting*. Después se aplica el posproceso descrito en la Sección 2.1: al menos una raíz, como

mucho cuatro etiquetas y las parejas alias siempre juntas.

Evaluación. Cada configuración se mide con la F1 macro de las predicciones *out-of-fold* sobre las 3 particiones de entrenamiento, acompañada de su desviación típica entre particiones. El *holdout* se reserva para medir una sola vez los modelos finales.

3.2 Búsqueda de hiperparámetros

El primer paso fue una búsqueda exhaustiva de los hiperparámetros anteriores combinados con tres tamaños de n-grama: (1,1), (1,2) y (1,3). En total se evaluaron 51 configuraciones. La Tabla 3.1 muestra la mejor de cada modelo y tamaño de n-grama, con el umbral por defecto y con el umbral por clase.

Tabla 3.1. Mejor configuración de cada modelo y tamaño de n-grama (F1 macro *out-of-fold* en entrenamiento).

Modelo	Hiperparámetros	n-gramas	Umbral por defecto	Umbral por clase
NB	*multinomial*, $\alpha = 0.01$	(1,1)	0'2727	0'2903 $\pm$ 0'0023
NB	*multinomial*, $\alpha = 0.01$	(1,2)	0'2158	0'2856 $\pm$ 0'0077
NB	*multinomial*, $\alpha = 0.01$	(1,3)	0'2148	0'2779 $\pm$ 0'0109
RL	$C = 2$	(1,1)	0'2992	0'3184 $\pm$ 0'0071
RL	$C = 8$	(1,2)	0'2402	0'3186 $\pm$ 0'0095
RL	$C = 32$	(1,3)	0'2109	0'3136 $\pm$ 0'0070
SVM	$C = 0.1$	(1,1)	0'2924	0'3194 $\pm$ 0'0074
SVM	$C = 0.3$	(1,2)	0'2295	**0'3233** $\pm$ 0'0057
SVM	$C = 0.3$	(1,3)	0'2175	0'3207 $\pm$ 0'0073

El umbral por clase es la decisión que más pesa mejorando todas las configuraciones. La mejor configuración, SVM con unigramas y bigramas y $C = 0.3$, alcanza una F1 macro de 0'3233. Aun así, el valor es modesto. La F1 macro promedia por igual las 28 categorías, y las minoritarias, con unas pocas decenas de ejemplos, se siguen reconociendo mal.

3.3 Técnicas para las clases minoritarias

Para intentar mejorar las clases pequeñas se diseñaron dos técnicas que aprovechan lo aprendido del análisis exploratorio.

Deduplicado inteligente. Los duplicados no pueden repartirse entre particiones, pero dentro del entrenamiento se puede decidir cuántas copias de cada artículo se conservan. Se comparan tres opciones: conservar todas las copias (`none`, 43.555 filas), una sola por artículo (`full`, 32.577) o el deduplicado inteligente (`smart`, 33.721). Este último conserva todas las copias de un artículo si tiene alguna categoría minoritaria (con menos de 1.000 artículos) y una sola si solo tiene categorías mayoritarias. Funciona como

un sobremuestreo ligero de las clases raras; por ejemplo, `cs.CL` pasa de 704 a 1.053 ejemplos y `cs.SI` de 263 a 428, mientras que las mayoritarias pierden las copias que solo inflaban su peso.

Enfoque jerárquico. Como todo artículo lleva una raíz y muchas secundarias la determinan, se entrenan clasificadores solo para las 25 categorías secundarias y las raíces se deducen de ellas. En cada partición se calcula $P(r \mid s)$ en entrenamiento, y una secundaria s asigna la raíz r si $P(r \mid s) \geq 0.5$. Para el 70'7 % de artículos que no tienen secundaria se entrena además un clasificador multiclase sobre las 7 combinaciones de raíces existentes. Se prueban dos formas de combinarlos: usar el modelo de raíces solo cuando no se predice ninguna secundaria (`fallback`) o unir las raíces de ambas fuentes (`union`).

La Tabla 3.2 cruza las tres opciones de deduplicado con el enfoque por defecto y el jerárquico (`fallback`). Cada modelo usa su mejor configuración de la Tabla 3.1.

Tabla 3.2. F1 macro *out-of-fold* según el tratamiento de duplicados y el enfoque.

	Por defecto			Jerárquico		
Modelo	`none`	`full`	`smart`	`none`	`full`	`smart`
NB	0'2862	0'2848	**0'2903**	0'2740	0'2732	0'2789
RL	0'3187	**0'3194**	0'3186	0'3067	0'3076	0'3073
SVM	0'3227	**0'3244**	0'3233	0'3114	0'3129	0'3123

Ninguna de las dos técnicas mejora el resultado. Las tres opciones de deduplicado difieren en menos de 0'006, dentro del ruido. Se usará deduplicado `smart` porque las diferencias son ruido y es la mejor opción en *Naive Bayes*. El enfoque jerárquico, en cambio, empeora todos los modelos en algo más de un punto, y la variante `union` tampoco lo corrige (SVM con 0'3179 frente a 0'3233). El diagnóstico en el *holdout* (variante `union`, SVM) explica por qué: las etiquetas secundarias predichas por error imponen raíces equivocadas. Las reglas asignan la raíz al 46'6 % de los artículos, cuando solo el 29'2 % tiene realmente una etiqueta secundaria. Así, la combinación de raíces es correcta en el 58'1 % de los artículos, frente al 69'4 % que logra el modelo de raíces por sí solo. Por ello se descarta el enfoque jerárquico.

3.4 Representaciones por debajo de la palabra

El último paso actuó sobre la representación. La tokenización por palabras trata como desconocido cualquier término que no haya visto, y el análisis exploratorio mostró que el 48 % de los tipos de palabra del test no aparece en entrenamiento. Se probaron dos alternativas:

- **n-gramas de caracteres.** Se añade al TF-IDF de palabras un segundo TF-IDF de n-gramas de 2 a 5 caracteres dentro de cada palabra (`analyzer="char_wb"`, `min_df=3`, 300.000 rasgos), y ambos se concatenan. Así se capturan raíces, sufijos y variantes

como *segment-*, *-graph* o *llms*.

- **Subpalabras WordPiece.** Se sustituye la tokenización por palabras por un vocabulario WordPiece entrenado sobre los propios textos, sin etiquetas, de modo que *segmentation* se divide en *segment* y *##ation*.

Con los n-gramas de caracteres se repitió la búsqueda completa de la Sección 3.2. La Tabla 3.3 compara el mejor resultado de cada modelo con y sin ellos.

Tabla 3.3. Mejor F1 macro *out-of-fold* de cada modelo con y sin n-gramas de caracteres.

Representación	NB	RL	SVM
Palabras	0'2903	0'3186	0'3233
Palabras + caracteres (2–5)	0'2942	0'3270	**0'3288**
Diferencia	+0'0039	+0'0084	+0'0055

Los n-gramas de caracteres mejoran los tres modelos y son la única técnica con una ganancia consistente. WordPiece, en cambio, no aporta mejora alguna en comparación con los n-gramas en ninguno de los casos.

3.5 Modelo final

El modelo seleccionado es la SVM lineal con TF-IDF de palabras (1,2) y caracteres (2–5), $C = 0.05$, `class_weight="balanced"`, deduplicado `smart`, enfoque por defecto y umbral por clase. Se elige por tener la mejor F1 macro *out-of-fold*, que es el criterio fijado de antemano. La Tabla 3.4 muestra el mejor modelo de cada tipo en el *holdout*, junto con una recreación del modelo *baseline* 1 del enunciado.

Tabla 3.4. Resultados en el *holdout* (4.903 artículos) de los mejores modelos con palabras + caracteres.

Modelo	F1 macro	Precisión	Cobertura	Acc.	HL
Baseline 1 (MNB)	0'2202	0'1654	0'6408	0'1779	0'0686
NB	0'2937	0'2798	0'4302	0'3710	0'0358
RL	0'3236	0'2844	0'4361	0'3726	0'0355
SVM (seleccionado)	0'3198	0'2649	0'4975	0'3838	0'0352

Los tres modelos superan con claridad al *baseline* en todas las métricas. La regresión logística queda ligeramente por encima de la SVM en F1 macro, pero la diferencia (0'004) es menor que la variabilidad entre particiones (k-folds), y la SVM tiene mejor *accuracy* y *Hamming loss*. Para el envío a *Codabench*, la SVM se reentrena con todos los artículos (entrenamiento y *holdout*) con los mismos hiperparámetros, y se aplican los umbrales ajustados con las predicciones *out-of-fold* de entrenamiento.

Resultado en *Codabench*. Este modelo fue el primer envío a la competición. La Tabla 3.5 compara su resultado en el test con el del *holdout* y con el *baseline* 1 del enunciado.

Tabla 3.5. SVM seleccionada en el *holdout* y en el test de *Codabench*, frente al *baseline* 1 en el test.

	F1 macro	Precisión	Cobertura	Acc.	HL
Baseline 1 (test)	0'143	0'113	0'495	0'111	0'084
SVM (*holdout*)	0'3198	0'2649	0'4975	0'3838	0'0352
SVM (test)	0'1906	0'1603	0'3203	0'2395	0'0455

En el test, la F1 macro baja de 0'3198 a 0'1906, y todas las métricas empeoran en una proporción parecida. Esta caída es la esperada por el desplazamiento entre entrenamiento y test descrito en la Sección 2.1: el modelo se evalúa sobre artículos más recientes, con terminología que no ha visto. Aun así, supera al *baseline* 1 en F1 macro (0'1906 frente a 0'143, un 33 % más), en *accuracy* (0'2395 frente a 0'111), en precisión y en *Hamming loss*. Solo queda por debajo en cobertura, porque el *baseline* predice muchas más etiquetas por artículo.

04

Modelos basados en Transformers

Los modelos *Transformer* se evaluaron con el mismo protocolo que los clásicos, las mismas particiones, el umbral por clase con *cross-fitting* y el mismo posproceso. Así sus cifras son directamente comparables con las de la SVM. La experimentación siguió cuatro pasos: representaciones congeladas, comparación de modelos preentrenados con ajuste fino, ablaciones sobre los tres mejores y, por último, entrenamiento completo con las 3 particiones de las mejores variantes.

4.1 Configuración común

Modelo. Cada encoder preentrenado recibe como entrada la concatenación `título. resumen`. Sobre el vector del primer token (`[CLS]`), que actúa como representación global de todo el documento, se coloca una cabeza clasificadora con *dropout* de 0'1 y una capa lineal de 28 salidas con activación sigmoide. Para que el modelo parta de la frecuencia real de cada clase, el sesgo de cada salida se inicializa a $\log \frac{p_j}{1-p_j}$, donde p_j es la proporción de artículos de la categoría j.

Entrenamiento. Se usa entropía cruzada binaria, AdamW con tasa de aprendizaje 3×10^{-5} para el encoder y diez veces mayor para la cabeza, *weight decay* de 0'01. El lote efectivo es de 32, se entrenan 4 épocas y se conserva el modelo de la última época, porque elegir la mejor época con la partición de validación inflaría su F1. Cada artículo entra una sola vez en el entrenamiento (deduplicado `full`), y la semilla se fija.

Evaluación. Ajustar un *Transformer* cuesta unos 20 minutos por partición, así que la comparación de modelos y las ablaciones se hacen solo sobre la primera partición. Sus umbrales se ajustan por *cross-fitting* entre las dos mitades de su validación, y la SVM se mide de la misma forma como referencia (F1 macro de 0'3145). Solo los modelos finales se entrenan con las 3 particiones, lo que da una F1 macro *out-of-fold* comparable con la del capítulo anterior.

4.2 Representaciones congeladas

La opción más barata es no ajustar el encoder, es decir, cada texto se convierte una vez en un vector normalizado y sobre él se entrena un clasificador. Se probaron una regresión logística por clase (`class_weight="balanced"`, $C \in \{0.1, 1\}$) y un kNN que puntúa cada clase

con las etiquetas de los 25 vecinos más similares (similitud coseno). Se compararon cuatro modelos de *embeddings*: SPECTER2, entrenado con artículos científicos y sus citas; BGE-large y E5-large, de propósito general; y Qwen3-Embedding-0.6B, de 2025, cuyo vocabulario ya cubre la terminología reciente.

Tabla 4.1. F1 macro *out-of-fold* (3 particiones) con representaciones congeladas.

Modelo de *embeddings*	Regresión logística	kNN
SPECTER2 (`allenai/specter2_base`)	**0'3134**	0'3132
BGE-large (`BAAI/bge-large-en-v1.5`)	0'2848	0'3107
E5-large (`intfloat/e5-large-v2`)	0'2862	0'2980
Qwen3-Embedding (`Qwen/Qwen3-Embedding-0.6B`)	0'2992	0'3009
SVM (Capítulo 3)	0'3288	

Ninguna representación congelada alcanza a la SVM (Tabla 4.1). La mejor es SPECTER2, la única especializada en artículos científicos, lo que indica que el dominio del preentrenamiento importa más que el tamaño o la actualidad del modelo.

4.3 Ajuste fino de modelos preentrenados

Se ajustaron seis modelos con la misma receta. Cinco son encoders de tamaño base: RoBERTa, el del *baseline* 2, SciBERT, preentrenado con artículos científicos, SPECTER2, que parte de SciBERT y se reentrena con el grafo de citas, DeBERTa-v3, un encoder generalista fuerte en clasificación, y ModernBERT, un encoder generalista de finales de 2024. El sexto es Qwen3-Embedding-0.6B, el modelo de 2025 de la Sección 4.2, ahora ajustado. Es un decoder de 600 millones de parámetros que es un tamaño demasiado grandre para ejecutar un ajuste completo, así que se ajusta con LoRA (una técnica eficiente que entrena solo un pequeño grupo de parámetros para ahorrar memoria) y el documento se representa con el último token.

Tabla 4.2. Ajuste fino (F1 macro en la primera partición).

Modelo	F1 macro
SVM (referencia)	0'3145 $\pm$ 0'0087
DeBERTa-v3 (`microsoft/deberta-v3-base`)	0'2688 $\pm$ 0'0017
RoBERTa (`roberta-base`)	0'3201 $\pm$ 0'0003
ModernBERT (`answerdotai/ModernBERT-base`)	0'3238 $\pm$ 0'0096
SciBERT (`allenai/scibert_scivocab_uncased`)	0'3394 $\pm$ 0'0082
Qwen3-Embedding + LoRA (`Qwen/Qwen3-Embedding-0.6B`)	0'3428 $\pm$ 0'0079
SPECTER2 (`allenai/specter2_base`)	**0'3473** $\pm$ 0'0128

Salvo DeBERTa-v3, que queda por debajo de la SVM con la receta común, el ajuste fino supera a la SVM con todos los modelos (Tabla 4.2). La diferencia solo es clara con los modelos preentrenados sobre textos científicos y con Qwen3. Este último gana unos

4'5 puntos respecto a su versión congelada, pero no supera a SPECTER2. SPECTER2 obtiene el mejor resultado. Los tres mejores modelos (SPECTER2, Qwen3 con LoRA y SciBERT) pasan a la fase de ablaciones.

4.4 Ablaciones

Para medir el efecto de cada decisión de diseño, sobre los tres mejores modelos del ajuste fino se probaron, de una en una, cuatro modificaciones (Tabla 4.3):

- **`pos_weight`**: entropía cruzada binaria con un peso $\sqrt{n_{\text{neg}}/n_{\text{pos}}}$ (entre 1 y 30) en los positivos de cada clase, para dar más importancia a las clases raras.
- **LoRA**: el encoder se congela y solo se entrenan matrices de rango 16 añadidas a sus capas lineales (un 2'4 % de los parámetros), con tasa de aprendizaje 3×10^{-4}. No se aplica a Qwen3, que ya se ajusta así.
- **LLRD** (*layer-wise learning rate decay*): la tasa de aprendizaje se multiplica por 0'9 en cada capa hacia abajo (0'95 en Qwen3, que tiene más capas), para alterar menos las capas inferiores, más generales.
- **cRT** (*classifier re-training*): tras el ajuste fino, la cabeza se sustituye por una regresión logística balanceada entrenada sobre los vectores `[CLS]`. Solo se probó con SPECTER2.

Tabla 4.3. Ablaciones sobre los tres mejores modelos (F1 macro en la primera partición). La desviación entre mitades está entre 0'002 y 0'020.

Variante	SPECTER2	Qwen3 + LoRA	SciBERT
Referencia	0'3473	0'3428	0'3394
+ `pos_weight`	0'3476	**0'3652**	0'3433
+ LoRA	**0'3634**	—	**0'3583**
+ LLRD	0'3515	0'3550	0'3397
+ cRT	0'2978	—	—

LoRA mejora claramente a los dos encoders (unos 1'6 puntos en SPECTER2 y 1'9 en SciBERT), y `pos_weight` mejora a Qwen3 en más de 2 puntos, aunque apenas cambia a los otros dos. LLRD da mejoras pequeñas, dentro del ruido. El cRT empeora claramente: la cabeza aprendida durante el ajuste fino funciona mejor que una regresión logística reentrenada.

4.5 Modelos finales

Las mejores variantes de cada modelo en la primera partición se entrenaron con las 3 particiones: en SPECTER2, las tres ablaciones que superaban a la referencia (LoRA, LLRD y `pos_weight`), en SciBERT, LoRA, y en Qwen3, `pos_weight`. La predicción del test de cada modelo es la media de los tres modelos entrenados, uno por partición.

Tabla 4.4. Modelos finales: F1 macro *out-of-fold* (3 particiones) y F1 macro en el *holdout*. Precisión, cobertura, *accuracy* y *Hamming loss* corresponden a las predicciones *out-of-fold*.

Modelo	F1 CV	Prec.	Cob.	Acc.	HL	F1 *holdout*
SVM	0'3288 ± 0'0049	0'2832	0'4406	0'4041	0'0330	0'3277
SPECTER2	0'3481 ± 0'0055	0'2999	0'4758	0'4373	0'0312	0'3497
SPECTER2 + `pos_weight`	0'3505 ± 0'0043	0'3085	0'4297	0'4319	0'0307	0'3651
SPECTER2 + LoRA	0'3444 ± 0'0080	0'2911	0'4752	0'4371	0'0309	0'3686
SPECTER2 + LLRD	0'3519 ± 0'0082	0'3054	0'4793	0'4395	0'0304	0'3665
SciBERT + LoRA	0'3488 ± 0'0068	0'3112	0'4519	0'4343	0'0310	0'3595
Qwen3 + LoRA + `pos_weight`	**0'3643** ± 0'0018	0'3157	0'4517	0'4300	0'0308	**0'3879**

Con las 3 particiones, todos los *Transformers* superan a la SVM en F1 macro, precisión, *accuracy* y *Hamming loss* (Tabla 4.4). La mejora se concentra en las clases minoritarias y temáticas: con SPECTER2, la F1 de `cs.CR` sube de 0'437 a 0'532, la de `stat.AP` de 0'068 a 0'112, la de `cs.IT` y `math.IT` de 0'096 a 0'142 y la de `cs.CL` de 0'468 a 0'509, mientras que `cs.CV` y `cs.LG` apenas cambian.

Las variantes de SPECTER2 y SciBERT quedan a menos de 1 punto entre sí, dentro de la desviación entre particiones. La ventaja de LoRA en la primera partición desaparece con las 3, lo que confirma que una sola partición no basta para decidir. Qwen3 con LoRA y `pos_weight` es la única variante que se separa, supera a la mejor de SPECTER2 en más de 1 punto de F1 macro *out-of-fold*, tiene la menor desviación entre particiones y también es la mejor en el *holdout*.

Resultados en *Codabench*. Se enviaron dos *Transformers*, ambos con los umbrales ajustados sobre todas las predicciones *out-of-fold* de desarrollo. El primero fue SPECTER2 con LLRD, el mejor modelo según la F1 *out-of-fold* antes de completar las ablaciones de Qwen3. El segundo fue Qwen3 con LoRA y `pos_weight`, el mejor una vez completadas. La Tabla 4.5 los compara en el test con la SVM y con los dos *baselines*.

Tabla 4.5. Resultados en el test de *Codabench*.

Modelo	F1 macro	Precisión	Cobertura	Acc.	HL
Baseline 1 (Count 5k + MNB)	0'143	0'113	0'495	0'111	0'084
Baseline 2 (RoBERTa)	0'138	0'145	0'160	0'371	0'034
SVM	0'1906	0'1603	0'3203	0'2395	0'0455
SPECTER2 + LLRD	0'1970	0'1725	0'3062	0'2426	0'0454
Qwen3 + LoRA + `pos_weight`	**0'2108**	0'1889	0'3483	0'2188	0'0459

En el test, los tres modelos pierden entre un 42 y un 46 % de su F1 macro respecto al *holdout*, como cabía esperar por el desplazamiento entre entrenamiento y test. El orden de desarrollo se mantiene, y Qwen3 con LoRA y `pos_weight` es el mejor envío (0'2108). Es el modelo más reciente y el único que ya conoce la terminología nueva del test, lo que puede explicar que conserve mejor su ventaja. Frente al *baseline* 2, la F1 macro

mejora un 53 %. El *baseline* 2 tiene más *accuracy* y menos *Hamming loss* porque predice muy pocas etiquetas (cobertura de 0’160), justo el comportamiento que la F1 macro penaliza.

05

Combinación de modelos

El último paso fue combinar los modelos ya entrenados. No se entrena ningún modelo nuevo: se reutilizan sus puntuaciones *out-of-fold* en desarrollo y sus puntuaciones en *holdout* y test. Por eso cualquier combinación cuesta segundos o pocos minutos en CPU. La idea es que modelos distintos (una SVM sobre n-gramas, encoders científicos y un modelo reciente como Qwen3) pueden equivocarse en artículos distintos, y su combinación puede corregir parte de esos errores.

5.1 Protocolo

Normalización. Cada modelo da puntuaciones en una escala distinta: *logits* los *Transformers*, distancias al hiperplano la SVM y fracciones de vecinos el kNN. Antes de combinarlas, cada columna (modelo y clase) se estandariza con la media y la desviación de sus puntuaciones *out-of-fold*, y la misma transformación se aplica al *holdout* y al test.

Evaluación anidada. Algunos combinadores aprenden de las etiquetas, ya sea unos pesos o un metaclasificador. Si se ajustaran con las mismas filas con las que se evalúan, su F1 saldría inflada. Por eso el combinador también se ajusta por *cross-fitting*. Después se ajustan los umbrales por clase y se aplica el posproceso como en los capítulos anteriores. Para el test, el combinador se ajusta con todas las predicciones *out-of-fold* de desarrollo. Solo se combinan modelos entrenados con las 3 particiones, de modo que la F1 macro *out-of-fold* de un *ensemble* es directamente comparable con la de un modelo individual.

5.2 Métodos y grupos de modelos

Se compararon seis formas de combinar las puntuaciones:

- **Media**: media simple de los modelos, sin ajuste.
- **Media ponderada**: los pesos se obtienen por selección voraz (Caruana *et al.*, 2004). Se parte del mejor modelo y, en cada uno de 20 pasos, se añade (con repetición) el que más sube la F1 macro. El peso de cada modelo es la fracción de veces que se ha elegido.
- **Votos**: cada modelo decide con sus propios umbrales, la puntuación de una etiqueta

es la fracción de modelos que la predicen, y después se ajusta un umbral por clase.

- ***Stacking* por clase**: una regresión logística por categoría ($C = 1$) que recibe las puntuaciones de todos los modelos para esa categoría. Equivale a unos pesos distintos para cada clase.
- ***Stacking* con todas las clases**: una regresión logística por categoría ($C = 0.05$) que recibe las puntuaciones de todos los modelos para las 28 categorías, de modo que puede aprender relaciones entre etiquetas.
- ***Stacking* con LightGBM**: igual que el anterior, pero con un *boosting* de árboles por categoría (300 árboles, tasa de aprendizaje 0'03 y 15 hojas), capaz de captar interacciones no lineales.

Estos métodos se aplicaron a cinco grupos de modelos, de menor a mayor tamaño:

- **SPECTER2 + SVM** (3 modelos): SPECTER2 con LLRD, SPECTER2 con `pos_weight` y la SVM.
- **Top 3** (3 modelos): el mejor modelo de cada arquitectura, es decir, Qwen3 con LoRA y `pos_weight`, SPECTER2 con LLRD y SciBERT con LoRA.
- **Top 3 + SVM** (4 modelos): los anteriores más la SVM, que representa el texto de una forma muy distinta.
- **Top 6** (6 modelos): los anteriores más SPECTER2 con `pos_weight` y el kNN sobre SPECTER2 congelado.
- **Todo** (11 modelos): todos los modelos con las 3 particiones, es decir, las siete variantes de *Transformers*, la SVM y tres modelos sobre representaciones congeladas.

5.3 Resultados

Se evaluaron las 30 combinaciones de método y grupo (Tabla 5.1). La referencia es el mejor modelo individual, Qwen3 con LoRA y `pos_weight`, con una F1 macro *out-of-fold* de 0'3643 $\pm$ 0'0018.

Tabla 5.1. F1 macro *out-of-fold* (3 particiones) de cada combinación. La desviación entre particiones está entre 0'001 y 0'013. Referencia: mejor modelo individual, 0'3643.

Método	SPECTER2 + SVM	Top 3	Top 3 + SVM	Top 6	Todo
Media	0'3540	0'3662	0'3623	0'3623	0'3614
Media ponderada	0'3567	0'3628	0'3628	0'3617	**0'3668**
Votos	0'3547	0'3627	0'3648	0'3621	0'3589
Stacking por clase	0'3563	0'3640	0'3581	0'3615	0'3597
Stacking todas las clases	0'3460	0'3465	0'3471	0'3446	0'3283
Stacking LightGBM	0'3157	0'3090	0'3169	0'3251	0'3207

Las combinaciones simples empatan con el mejor modelo. Con los cuatro métodos simples (media, media ponderada, votos y *stacking* por clase), todos los grupos que incluyen a Qwen3 quedan entre 0'3581 y 0'3668, a menos de 1 punto del mejor modelo individual. Las dos mejores combinaciones, la media ponderada de todos los modelos (0'3668 $\pm$ 0'0042) y la media del top 3 (0'3662 $\pm$ 0'0060), solo superan a Qwen3 en 0'0025 y 0'0019, menos que la desviación entre particiones. En el *holdout* tampoco hay mejora: 0'3862 y 0'3853, frente a 0'3879 de Qwen3 solo. Sin Qwen3 (grupo SPECTER2 + SVM), ninguna combinación pasa de 0'3567: combinar modelos parecidos apenas mejora a sus miembros.

Añadir modelos más débiles no aporta. Ampliar el top 3 con la SVM, el kNN o más variantes de SPECTER2 no mejora la media, que baja de 0'3662 a 0'3614. Los pesos de la media ponderada lo confirman: en los grupos que incluyen a Qwen3, la selección voraz da peso cero a la SVM y a los modelos sobre representaciones congeladas, y con todos los modelos reparte el 39 % a Qwen3 y el resto entre cuatro variantes de SPECTER2 y SciBERT. Esto sugiere que los *Transformers* cometen errores parecidos entre sí, y que la SVM, aunque es distinta, es demasiado débil para aportar.

Los metaclasificadores complejos sobreajustan. Los *stackings* que reciben las 28 categorías de todos los modelos empeoran en todos los grupos: la regresión logística queda entre 1'7 y 3'6 puntos por debajo del mejor modelo individual, y LightGBM entre 3'9 y 5'5. Con decenas o cientos de entradas por categoría y clases con unas pocas decenas de positivos, el metaclasificador aprende ruido en lugar de relaciones entre etiquetas, y cuanto más flexible es, más empeora.

5.4 Resultado en *Codabench*

Se envió la combinación con mayor F1 macro *out-of-fold*, la media ponderada de todos los modelos, con los umbrales ajustados sobre todas sus predicciones *out-of-fold*. La Tabla 5.2 la compara con los envíos anteriores.

Tabla 5.2. Resultados en el test de *Codabench*.

Modelo	F1 macro	Precisión	Cobertura	Acc.	HL
SVM	0'1906	0'1603	0'3203	0'2395	0'0455
SPECTER2 + LLRD	0'1970	0'1725	0'3062	0'2426	0'0454
Qwen3 + LoRA + `pos_weight`	**0'2108**	0'1889	0'3483	0'2188	0'0459
Media ponderada (todo)	0'2079	0'1882	0'3256	0'2274	0'0454

En el test, la media ponderada (0'2079) queda ligeramente por debajo de Qwen3 solo (0'2108), lo que confirma que su ventaja en desarrollo era ruido. Además, el 61 % de su peso recae en variantes de SPECTER2 y SciBERT, que en el test generalizan peor que Qwen3, por lo que la combinación diluye la ventaja del único modelo que conoce la

terminología reciente. El mejor envío de la competición sigue siendo, por tanto, Qwen3 con LoRA y `pos_weight`.

06

Clasificación *few-shot* con un LLM

Como última prueba se evaluó un enfoque sin entrenamiento: un modelo de lenguaje de instrucciones recibe unos pocos artículos ya etiquetados como ejemplo y responde directamente con las categorías del artículo. Si funcionara, tendría una ventaja clara frente al desplazamiento temporal, porque no se ajusta a la distribución del entrenamiento.

6.1 Configuración

Modelo. Se usa Qwen3-8B sin modo de razonamiento, servido con vLLM en dos GPU T4 y con decodificación voraz (temperatura 0). El mensaje de sistema contiene las 28 categorías con su nombre oficial en *arXiv* y pide responder solo con los códigos, separados por comas.

Ejemplos por recuperación. Los k ejemplos ($k = 0, \ldots, 5$) no son fijos. Para cada artículo se eligen los k artículos de entrenamiento más parecidos según los *embeddings* de SPECTER2 (Sección 4.2), y el más parecido se coloca el último, justo antes de la pregunta. Cada ejemplo es un turno de usuario con título y resumen (recortado a 120 palabras) seguido de un turno del asistente con sus etiquetas reales. El resumen del artículo a clasificar se recorta a 300 palabras.

Decodificación. De la respuesta se extraen los códigos de categoría válidos en el orden en que aparecen, y se aplica el posproceso de siempre: como mucho cuatro etiquetas, alias juntos y al menos una raíz. Si falta la raíz, se añade la más votada entre los 10 vecinos más parecidos.

Evaluación. Como no hay nada que ajustar con etiquetas, ni pesos ni umbrales, no hace falta validación cruzada. Por el coste de la inferencia, cada valor de k se evalúa en un subconjunto de 1.009 artículos del *holdout*, con ejemplos extraídos solo de desarrollo.

6.2 Resultados

Los ejemplos son imprescindibles (Tabla 6.1). Sin ellos ($k = 0$), el modelo asigna más de tres etiquetas por artículo y casi nunca acierta el conjunto exacto (*accuracy* de 0'0614): conoce los temas, pero no el criterio con el que se etiquetó el corpus. Con ejemplos, la

Tabla 6.1. Clasificación *few-shot* en 1.009 artículos del *holdout* según el número de ejemplos k.

k	F1 macro	Precisión	Cobertura	Acc.	HL	Etiquetas/artículo
0	0'1818	0'1524	0'3220	0'0614	0'0806	3'25
1	0'2253	0'1955	0'3042	0'2121	0'0499	2'36
2	0'2431	0'2065	0'3206	0'2696	0'0460	2'31
3	0'2370	0'2008	0'3112	0'2567	0'0464	2'27
4	**0'2581**	0'2179	0'3375	0'2616	0'0429	2'21
5	0'2568	0'2170	0'3357	0'2656	0'0415	2'18

F1 macro sube hasta 0'2581 con $k = 4$ y se estanca a partir de ahí.

Aun así, queda muy lejos de los modelos ajustados, que obtienen entre 0'35 y 0'39 en el *holdout* completo. El LLM acierta las clases frecuentes (F1 de 0'919 en `cs.CV`), pero falla en las minoritarias porque aplica su propia idea de cada categoría. Por ejemplo, predice `cs.IR` 27 veces para 4 artículos reales y `cs.IT` 18 veces para uno solo.

Resultado en *Codabench*. Se envió la configuración con $k = 4$ en su variante sin raíz forzada, por si en los artículos recientes no se cumpliera la regla de la raíz. En el *holdout* ambas variantes son prácticamente idénticas (0'2582 frente a 0'2581), porque el modelo casi siempre incluye una raíz por sí mismo. La Tabla 6.2 la compara con los *baselines* y con el mejor envío.

Tabla 6.2. Resultados en el test de *Codabench*.

Modelo	F1 macro	Precisión	Cobertura	Acc.	HL
Baseline 1 (Count 5k + MNB)	0'143	0'113	0'495	0'111	0'084
Baseline 2 (RoBERTa)	0'138	0'145	0'160	0'371	0'034
Qwen3-8B *few-shot* ($k = 4$)	0'1634	0'1236	0'3403	0'1132	0'0592
Qwen3 + LoRA + `pos_weight`	**0'2108**	0'1889	0'3483	0'2188	0'0459

En el test, el *few-shot* obtiene una F1 macro de 0'1634. Supera a los dos *baselines*, pero queda casi 5 puntos por debajo del mejor modelo ajustado. Su caída respecto al *holdout* es algo menor que la de los modelos ajustados (un 37 % frente a un 46 %), pero no compensa su peor punto de partida. Además, en el test predice 2'64 etiquetas por artículo y asigna `cs.LG` al 78 % de los artículos, frente al 53 % del entrenamiento, lo que hunde su precisión. Con pocos ejemplos, el LLM no llega a aprender las convenciones de etiquetado del corpus, que los modelos ajustados sí capturan con miles de artículos.

07

Resultados y conclusiones

7.1 Resumen de resultados

La Tabla 7.1 reúne los cinco envíos a *Codabench* y los dos *baselines*, ordenados por su F1 macro en el test. Para cada uno se muestran las tres estimaciones disponibles: la F1 macro *out-of-fold* en desarrollo, que fue el criterio de selección; la F1 macro en el *holdout*; y las métricas en el test.

Tabla 7.1. Envíos a *Codabench* y *baselines*, ordenados por F1 macro en el test. Precisión, cobertura, *accuracy* y *Hamming loss* corresponden al test.

		F1 macro						
	Modelo	CV	*Holdout*	Test	Prec.	Cob.	Acc.	HL
1	Qwen3 + LoRA + `pos_weight`	0'3643	**0'3879**	**0'2108**	**0'1889**	0'3483	0'2188	0'0459
2	Media ponderada (11 modelos)	**0'3668**	0'3862	0'2079	0'1882	0'3256	0'2274	0'0454
3	SPECTER2 + LLRD	0'3519	0'3665	0'1970	0'1725	0'3062	0'2426	0'0454
4	SVM (palabras + caracteres)	0'3288	0'3198	0'1906	0'1603	0'3203	0'2395	0'0455
5	Qwen3-8B *few-shot* ($k = 4$)	—	0'2581†	0'1634	0'1236	0'3403	0'1132	0'0592
–	*Baseline* 1 (Count 5k + MNB)	—	0'2202	0'143	0'113	**0'495**	0'111	0'084
–	*Baseline* 2 (RoBERTa)	—	—	0'138	0'145	0'160	**0'371**	**0'034**

El mejor modelo. El mejor envío es Qwen3-Embedding-0.6B ajustado con LoRA y pérdida `pos_weight`, con una F1 macro de 0'2108 en el test. Supera al *baseline* 1 en un 47 % y al *baseline* 2 en un 53 %. Los cuatro modelos ajustados superan a ambos *baselines* en F1 macro, y el orden entre ellos coincide con el de la validación.

Desarrollo frente a test. Todos los modelos pierden entre un 37 y un 46 % de su F1 macro al pasar del *holdout* al test. Es la consecuencia del desplazamiento temporal detectado en el análisis exploratorio: el test contiene artículos más recientes, con terminología que no aparece en el entrenamiento. La validación local ha servido para ordenar los modelos, pero no para predecir su valor absoluto en la competición.

F1 macro frente a las demás métricas. El *baseline* 2 tiene la mejor *accuracy* y *Hamming loss* porque predice muy pocas etiquetas (cobertura de 0'160), mientras que el *baseline* 1 tiene la mayor cobertura porque predice demasiadas. Nuestros modelos, con umbrales

ajustados para la F1 macro, buscan un equilibrio entre ambos extremos y predicen algo más de dos etiquetas por artículo.

7.2 Conclusiones

Qué ha funcionado.

- **El umbral por clase** es la decisión más rentable de todo el trabajo. Aporta entre 2 y 10 puntos de F1 macro a los modelos clásicos, a coste casi nulo, y se ha aplicado a todos los modelos.
- **Un protocolo de validación cuidadoso**, con deduplicado, estratificación iterativa y *cross-fitting* de umbrales y combinadores, ha permitido comparar todos los modelos en igualdad de condiciones. También ha evitado decisiones erróneas, como elegir SPECTER2 con LoRA por su buen resultado en una sola partición, que desaparecía con las tres.
- **El preentrenamiento adecuado** importa más que la arquitectura. Los modelos científicos (SPECTER2, SciBERT) superan a los generalistas, y el modelo más reciente (Qwen3), que ya conoce parte de la terminología del test, es el mejor en la competición.
- **Los n-gramas de caracteres** en el modelo clásico y **LoRA** y `pos_weight` en los *Transformers* dan mejoras pequeñas pero consistentes.

Qué no ha funcionado. El deduplicado inteligente y el enfoque jerárquico no mejoran el modelo clásico. Las combinaciones de modelos empatan con el mejor individual y los metaclasificadores complejos sobreajustan. El *few-shot* con un LLM no aprende las convenciones de etiquetado del corpus a partir de unos pocos ejemplos.

Trabajo futuro. La principal limitación es el desplazamiento entre entrenamiento y test, así que las líneas más prometedoras atacan directamente ese problema:

- Continuar el preentrenamiento del encoder, sin etiquetas, sobre los resúmenes del test (adaptación al dominio).
- Ajustar con LoRA modelos recientes de mayor tamaño, que ya conozcan la terminología actual.
- Construir un conjunto de validación más parecido al test, con los artículos de entrenamiento que el clasificador adversarial confunde con artículos del test.